LAB 7

Fuzzing for Input Validation Bugs

Tennis Scoring System | Hypothesis Property-Based Testing

Student: Ahan Charan	**SRN:** PES2UG24CS037
Section: A	Repository: ahan-charan/SE-lab6-lab7-37

1. Objective

The objective of Lab 7 is to use fuzz testing with the Hypothesis library to discover input-validation bugs in the assigned Python application, analyse the resulting failures, and improve the application so unexpected or malformed input is handled safely.

2. Environment and tools

Item	Recorded environment
Operating environment	Windows Terminal running Ubuntu through WSL
Python	3.14.4
pytest	9.2.0
Hypothesis	6.170.15
Test command	pytest -v test_tennis_scoring.py

3. Functions under test

Function	Purpose
parse_score(text)	Parse a score such as 3-2 into a pair of integers.
game_lead(text)	Calculate the difference between two valid tennis point values (0, 15, 30, 40).
set_spread(games_csv)	Calculate the maximum minus minimum value in a comma-separated list of game scores.

4. Method

- Created Hypothesis tests using text and None strategies to exercise each function with unexpected inputs.
- Ran the three no-crash fuzz tests against the original implementation and recorded the failures and minimized examples.
- Added defensive input validation and then reran the complete test file, including property tests for valid inputs.

5. Bugs discovered in the original application

The first fuzz run selected the three no-crash tests. All three failed against the original implementation. Hypothesis reported multiple distinct counterexamples for each affected function.

Function	Hypothesis examples	Observed failure types
parse_score	'-', '', None	ValueError; AttributeError
game_lead	'-', '', None	KeyError; ValueError; AttributeError
set_spread	'', None	ValueError; AttributeError

6. Application changes

- Added a shared string-validation helper that rejects non-string and empty input.
- Added safe parsing for non-negative integer tokens, including validation before conversion.
- Updated parse_score to require exactly two valid integer components separated by a hyphen.
- Updated game_lead to accept only the tennis point values 0, 15, 30 and 40.
- Updated set_spread to reject empty CSV elements and invalid or negative values.
- Updated the interactive main function to display clear validation messages instead of terminating on invalid values.

7. Verification after the fix

Verification	Input	Observed result
Valid parse_score	3-2	(3, 2)
Valid game_lead	40-30	1
Valid set_spread	6,4,7	3
Invalid parse_score	abc	Validation message; no crash
Invalid game_lead	50-30	Validation message; no crash
Invalid set_spread	6,,7	Validation message; no crash

FINAL TEST RESULT: 6 PASSED

3 fuzz/no-crash tests + 3 valid-input property tests

8. Evidence note

The screenshots supplied for this report document the Hypothesis failures, the passing test run, and valid/invalid execution of the corrected application. **The handout also asks for a screenshot of the original application run before fuzz testing (Step 2). That specific screenshot was not among the images supplied, so it is not represented here. Capture and add it if your instructor requires evidence for every numbered step.**

Repository: https://github.com/ahan-charan/SE-lab6-lab7-37

Evidence 1 - Hypothesis failures in parse_score

Hypothesis minimized the failures to inputs such as an empty string, a single hyphen, and None. The original parser attempted to split and convert these values without validating the input.

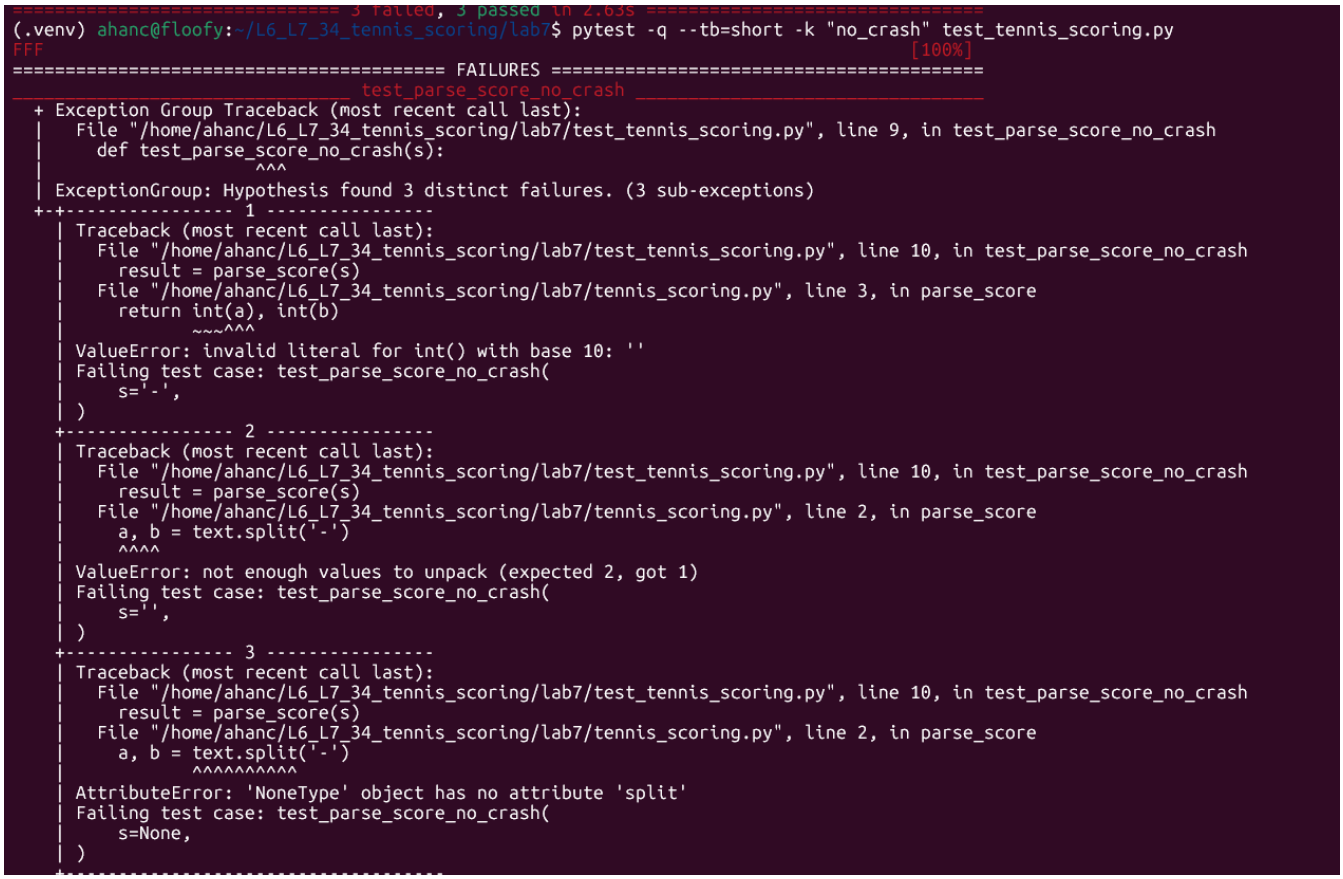


Figure: Evidence 1 - Hypothesis failures in parse_score.

Evidence 2 - Hypothesis failures in game_lead

Hypothesis exposed KeyError, ValueError, and AttributeError cases in the original game_lead function, including empty and missing score components.

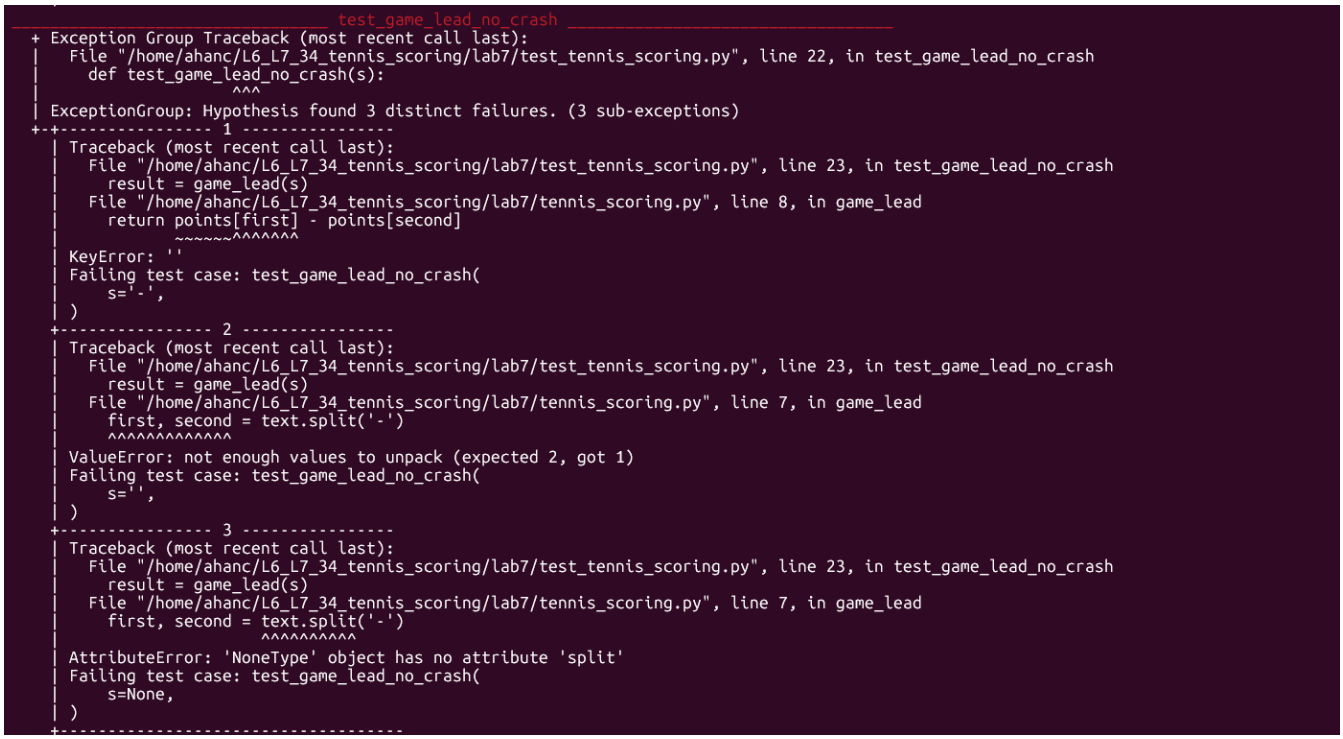


Figure: Evidence 2 - Hypothesis failures in game_lead.

Evidence 3 - Hypothesis failures in set_spread

Hypothesis found that empty CSV input and None caused the original set_spread function to raise ValueError and AttributeError. The terminal summary shows all three no-crash fuzz tests failing against the original implementation.

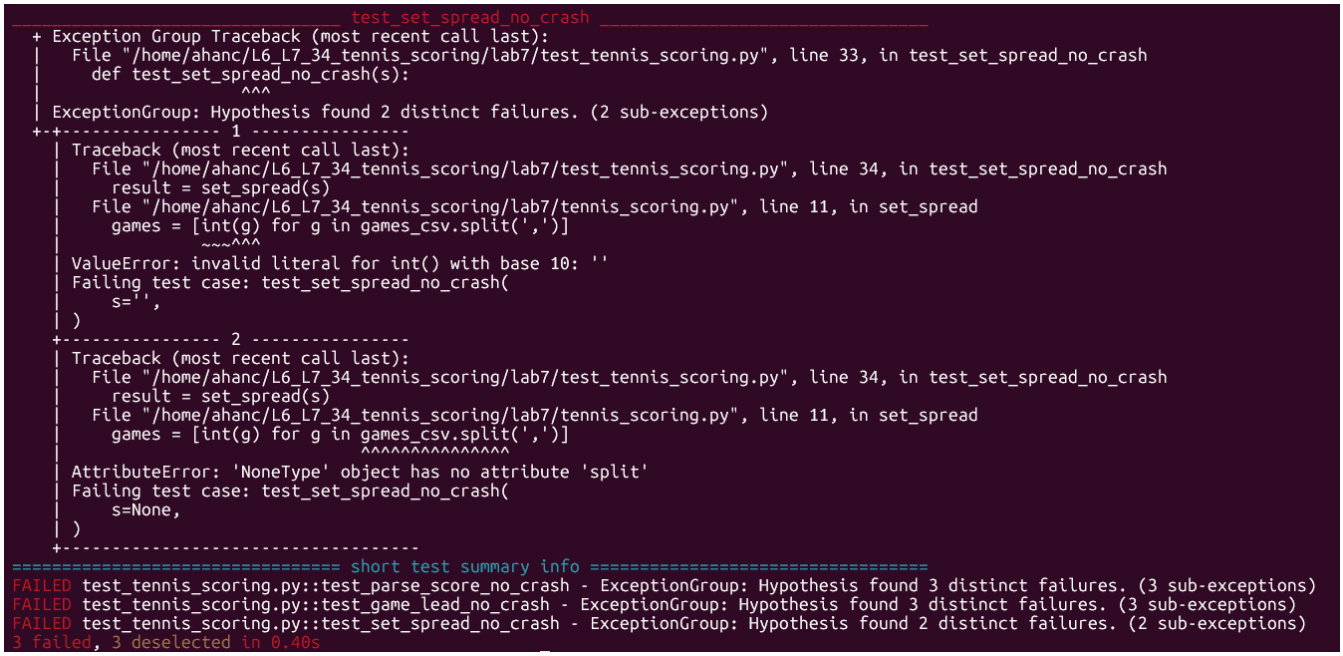


Figure: Evidence 3 - Hypothesis failures in set_spread.

Evidence 4 - All property-based tests pass after the fix

After input validation was added, pytest collected and passed all six tests: three fuzz/no-crash properties and three valid-input properties.

```
(.venv) ahanc@floofy:~/L6_L7_34_tennis_scoring/lab7$ nano tennis_scoring.py
(.venv) ahanc@floofy:~/L6_L7_34_tennis_scoring/lab7$ pytest -v test_tennis_scoring.py
============================= test session starts ==============================
platform linux -- Python 3.14.4, pytest-9.2.0, pluggy-1.7.0 -- /home/ahanc/L6_L7_34_tennis_scoring/lab6/.venv/bin/python
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: /home/ahanc/L6_L7_34_tennis_scoring/lab7
plugins: hypothesis-6.170.15
collected 6 items

test_tennis_scoring.py::test_parse_score_no_crash PASSED
test_tennis_scoring.py::test_game_lead_no_crash PASSED
test_tennis_scoring.py::test_set_spread_no_crash PASSED
test_tennis_scoring.py::test_parse_score_valid_inputs PASSED
test_tennis_scoring.py::test_game_lead_valid_inputs PASSED
test_tennis_scoring.py::test_set_spread_valid_inputs PASSED

============================== 6 passed in 0.54s ===============================
(.venv) ahanc@floofy:~/L6_L7_34_tennis_scoring/lab7$
```

Figure: Evidence 4 - All property-based tests pass after the fix.

Evidence 5 - Fixed application with valid inputs

The corrected application preserves expected behavior: 3-2 parses to (3, 2), 40-30 produces a lead of 1, and 6,4,7 produces a spread of 3.

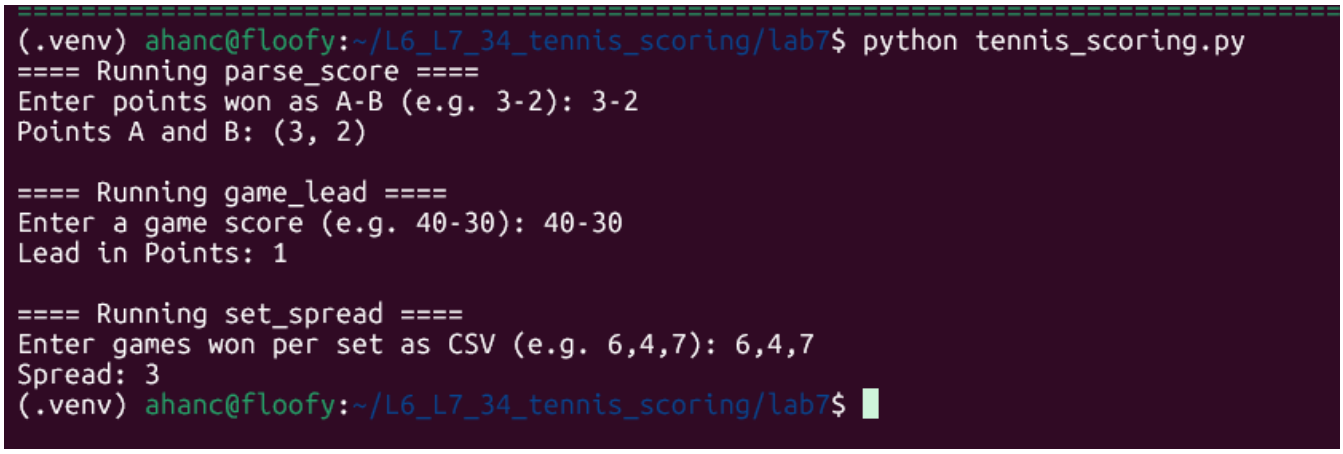


Figure: Evidence 5 - Fixed application with valid inputs.

Evidence 6 - Fixed application safely rejects invalid inputs

The corrected application displays validation messages for abc, 50-30, and 6,,7 and returns to the command prompt without crashing.

```
(.venv) ahanc@floofy:~/L6_L7_34_tennis_scoring/lab7$ python tennis_scoring.py
==== Running parse_score ====
Enter points won as A-B (e.g. 3-2): abc
Invalid score. Enter two non-negative integers separated by '-'.

==== Running game_lead ====
Enter a game score (e.g. 40-30): 50-30
Invalid game score. Use 0, 15, 30 or 40.

==== Running set_spread ====
Enter games won per set as CSV (e.g. 6,4,7): 6,,7
Invalid set scores. Enter comma-separated non-negative integers.
(.venv) ahanc@floofy:~/L6_L7_34_tennis_scoring/lab7$
```

Figure: Evidence 6 - Fixed application safely rejects invalid inputs.